

PERSONAL PROJECT REPORT

MindGrid

A Defense-in-Depth DevSecOps Project

Designing and Implementing a Hardened, Multi-Layer
Secure System Aligned with OWASP and CIS Benchmarks

Author

Yassine Ben Ayed

Date

June - August 2026

This document describes the design, implementation, and validation of a three-layer defense-in-depth security architecture, inspired by the March 2026 Trivy supply-chain attack.

Contents

1	General Introduction	1
1.1	Security Layers Overview	1
2	The Trivy Supply Chain Attack	3
2.1	What Does Trivy Actually Do in a Pipeline?	3
2.2	How the Attackers Broke In	3
2.3	What Was Actually Stolen	3
2.4	Remediation Measures	4
2.5	Applying These Lessons to This Project	4
3	Layer 1: Infrastructure / OS Hardening	6
3.1	Securing Lynis Itself	6
3.1.1	Step 1: Pin to an Exact, Verifiable Version, and Verify with GPG	6
3.1.2	Step 2: Verify Integrity Over Time, Not Just at Install	6
3.1.2.1	Limitation of This Approach	7
3.2	Network Namespace Isolation	7
3.2.1	Why Root Privileges Don't Undermine This Control	8
3.3	Scan Results	8
3.3.1	Automation Scripts Overview	9
3.4	Automated Security Patching: Unattended Upgrades	9
3.5	Fail2Ban: Brute-Force Protection	9
3.6	kube-bench: Kubernetes Cluster Hardening Audit	10
3.6.1	Scan Results	10
3.6.2	Remediation of kube-bench Findings	10
3.7	Verifying TLS Client Certificate Enforcement	10
3.7.1	Step 1: Test etcd Reachability from Another Machine on the Network	10
3.7.2	Step 2: Test etcd API Call	11
3.8	Addressing Residual Network Exposure	11
3.8.1	Is an Open, Reachable Port Actually a Problem Here?	11
3.8.2	Why It Still Matters	11
3.8.3	Remediation: Restricting etcd Access at the Network Layer	12
4	Layer 2: Application Workload Security	13
4.1	Application Overview	13
4.1.1	Communication	13
4.1.2	Storage	14
4.1.3	Secrets Management	14

4.1.4	Authentication and Authorization	14
4.1.5	Intentionally Introduced Vulnerabilities	14
4.2	MindGrid Application Architecture Overview	15
4.2.1	Containerization Approach	15
4.2.2	Choosing nginx-unprivileged for the Frontend	16
4.3	Cluster Node Architecture: Environment Isolation	16
4.3.1	Node Labeling and Tainting	16
4.3.2	Granting Access via Tolerations	16
4.3.3	Automated Environment Routing with Kyverno	17
4.4	Supporting Tooling	17
4.5	Scope of Hardening	18
4.6	Kyverno Policy Suite	18
4.6.1	A Necessary Exception: Kyverno's Own Controllers	19
4.7	Kubernetes Deployment Architecture	19
4.7.1	Overall Architecture	19
4.7.2	Gateway API Layer	21
4.7.3	Secret and Configuration Management	21
4.7.4	Storage Layer	22
4.8	From Working to Applying OWASP-Aligned Security Best Practices	23
4.8.1	Rate Limiting on Login	24
4.9	Security Baseline (Applied to All Deployments)	24
4.9.1	Non-Root Execution	25
4.9.2	Read-Only Root Filesystem	25
4.9.3	Capability Dropping	26
4.9.4	Seccomp Profile	26
4.9.5	Image Pinning by Digest	26
4.9.6	Liveness and Readiness Probes	26
4.10	Service Accounts and Identity	27
4.10.1	Verifying the Configuration	27
4.11	Network Policies	28
4.11.1	Design Philosophy: Default-Deny, Zero-Trust	28
4.11.2	Policy-by-Policy Breakdown	28
4.11.3	Connectivity Matrix	31
4.12	Kubescape: Runtime Compliance Scanning	32
4.12.1	The Question: Can Kubescape Exfiltrate Data?	32
4.12.2	Evaluating Candidate Controls	33
4.12.3	What Each Component Actually Needs	34
4.12.4	Final Compliance Score	34
5	Layer 3: CI/CD Security	35
5.1	Branching Strategy: Trunk-Based Development	35
5.2	The Deployment Flow	35
5.3	Securing the CI/CD Pipeline	36
5.3.1	Branch Protection Rules	36

5.3.2	GitHub Deployer: RBAC Identity for the Pipeline	37
5.4	The DevSecOps Pipeline	37
5.4.1	Pipeline Stages	38
5.4.2	Security Tools Summary	40
5.4.3	Key Security Design Points	40
5.5	Refining the ZAP Strategy: Authenticated vs. Unauthenticated Scanning	41
5.6	Scheduled Compliance Reporting	41
5.7	Nexus as a Dependency Proxy	41
6	Conclusion and Future Work	42

List of Figures

3.1	Lynis project folder structure: install-lynis.sh, run_lynis_scan.sh, auto-lynis.sh . . .	9
3.2	Overview of Layer 1: Infrastructure/OS hardening controls	12
4.1	MindGrid application architecture	15
4.2	Cluster overview: DevOps toolchain	18
4.3	Application traffic flow: Gateway to Services	20
4.4	Network policies: default-deny zero-trust architecture	32
5.1	Trunk-based development: short-lived branches merging frequently into main . .	35
5.2	DevSecOps pipeline diagram: full sequence from PR to production deployment : Part 1	39
5.3	DevSecOps pipeline diagram: full sequence from PR to production deployment : Part 2	39

List of Tables

3.1	What GPG verification protects against	6
3.2	Lynis scan: test count breakdown	8
3.3	Lynis automation scripts	9
4.1	MindGrid components	13
4.2	Cluster node roles	16
4.3	Supporting DevSecOps tooling	17
4.4	Kyverno policy files	19
4.5	Application workloads	20
4.6	Gateway path routing	21
4.7	ConfigMaps (non-sensitive)	22
4.8	Secrets (sensitive)	22
4.9	postgres-retain StorageClass properties	23
4.10	Rate limit policy parameters	24
4.11	Non-root execution settings	25
4.12	Writable scratch space exceptions	25
4.13	Service accounts	27
4.14	Gateway ingress policies	29
4.15	Network policy connectivity matrix	31
4.16	Candidate exfiltration controls evaluated	33
4.17	Kubescape component network requirements	34
4.18	Kubescape compliance scores	34
5.1	Deployment stages, triggers, and gates	36
5.2	Deployer Role permissions	37
5.3	Security tools used across pipeline stages	40

CHAPTER 1

General Introduction

This project was inspired by the **Trivy supply-chain attack of March 2026**. Following the disclosure of that incident, an investigation was carried out into exactly how the compromise occurred and what it revealed about weaknesses in modern CI/CD security practices, with the goal of translating those lessons into a concrete, hardened system rather than a purely theoretical study.

The objective of this project is to design and implement a comprehensive, defense-in-depth secure system, built in alignment with **OWASP** guidelines and **CIS** Benchmarks, and driven by security best practices at every layer of the stack.

To achieve this, the system is organized into multiple independent security layers, each addressing a distinct part of the infrastructure, so that the compromise of any single layer does not compromise the system as a whole.

NOTE Scope note: additional hardening, particularly runtime threat detection with Falco, centralized audit logging, and broader observability tooling, was identified as valuable but was not implemented due to resource and time constraints within the scope of this project. These are documented as future work rather than omissions.

1.1 Security Layers Overview

The system's defenses are structured into three complementary layers, each targeting a different stage of the infrastructure and software lifecycle.

Layer 1: Infrastructure / OS Hardening

Covers the security of the underlying hosts and cluster foundation: Ubuntu OS hardening, kubeadm cluster hardening, host-level auditing (**Lynis**), automated security patching, brute-force protection (**Fail2Ban**), and securing etcd (TLS client authentication, encryption at rest, network-level access restriction).

Layer 2: Application Workload Security

Covers the security of what runs inside the cluster: pod-level securityContext hardening, **RBAC** least-privilege access control, network policies, and policy enforcement via **Kyverno**.

Layer 3: CI/CD Security

Covers the security of the software supply chain itself: image vulnerability scanning, image signing, SBOM (Software Bill of Materials) generation and attestation, secrets management, branch protection rules, secret-leak detection, DAST, SAST, and SCA.

CHAPTER 2

The Trivy Supply Chain Attack

2.1 What Does Trivy Actually Do in a Pipeline?

When a company sets up CI/CD, Trivy is typically added as a step that automatically scans code and container images for vulnerabilities on every push. To perform this task, Trivy runs inside the pipeline, meaning it inherits access to everything that pipeline can reach: cloud credentials, GitHub tokens, database passwords, Kubernetes access, and more. Trivy itself doesn't need most of these permissions to function, but it executes in the same trusted context as tools that do.

2.2 How the Attackers Broke In

1. The attackers discovered a misconfiguration in Trivy's own GitHub Actions automation and exploited it to steal a secret token with permission to publish new Trivy releases.
2. Aqua Security, the company behind Trivy, detected the breach and rotated their secret keys, but the rotation was incomplete, and some of the compromised keys remained valid.
3. Three weeks later, the attackers returned and reused those leftover valid keys.
4. With this access, they exploited a subtle weakness in Git version tags: a tag such as `v0.35.0` is meant to permanently reference one specific commit, but tags can be force-moved to point to different code without changing the tag name itself. As a result, any pipeline referencing `trivy-action@v0.35.0` silently began pulling malicious code, with no visible change to its configuration.
5. The malicious payload was designed to execute first, silently exfiltrate all secrets available within the pipeline to an attacker-controlled server, erase its own traces, and then run the legitimate Trivy scan as normal, leaving the pipeline appearing fully green, with no errors or warnings.

2.3 What Was Actually Stolen

Any secret present in the pipeline at the moment the poisoned tool executed was at risk, including:

- GitHub tokens, enabling attackers to push code under a legitimate identity
- AWS and cloud credentials, enabling cloud account access
- Kubernetes tokens, enabling cluster access
- SSH keys
- Database passwords

2.4 Remediation Measures

- Fix 1** Upgrade to a clean version. Stop using the compromised release versions and migrate to versions confirmed unaffected.
- Fix 2** Never trust a tag; pin to a fixed commit SHA. This is the most critical lesson from the incident. A version tag can be reassigned by anyone with write access to the repository, which is exactly the mechanism the attackers exploited. A commit SHA, by contrast, is a fixed hash permanently bound to one specific, immutable piece of code and cannot be silently redirected.
- Fix 3** Block the attacker's exfiltration domain. Stolen data was sent to a typosquatted look-alike domain (aquasecurity.org, mimicking the legitimate aquasecurity.org). Blocking this domain at the network level prevents any lingering compromised systems from successfully exfiltrating data.
- Fix 4** Check for indicators of compromise. Look for the specific artifacts the attackers left behind, such as anomalous folder names or malicious files at known paths, to determine whether a given environment was affected.
- Fix 5** Assume compromise and rotate everything. If a pipeline executed during the vulnerable window, every secret it had access to should be treated as compromised and rotated immediately, without waiting for confirmation of theft.
- Fix 6** Don't trust cached artifacts. Systems that cache downloaded tools or images locally may retain the malicious version even after the upstream source has been cleaned. Local caches must therefore be invalidated or bypassed as part of remediation.

2.5 Applying These Lessons to This Project

Generalizing from this incident, the core defensive principles adopted throughout this project are: use immutable, commit-pinned versions wherever possible; isolate tools within their own network namespace; and apply the principle of least privilege consistently.

In practice, this proved significantly harder than expected. Not every tool supports immutable versioning, not every tool can operate fully offline, and several require elevated privileges to function. Applying this standard consistently across the seven to eight tools used in this project represented a substantial amount of work for a single person, but every tool was hardened to

the extent that was practically achievable.

CHAPTER 3

Layer 1: Infrastructure / OS Hardening

3.1 Securing Lynis Itself

Lynis carries a lower inherent risk than Trivy: it does not typically run inside a CI/CD pipeline with cloud secrets attached, it runs locally on a server to audit that server. However, it still warrants careful scrutiny, since it executes with elevated (root) privileges in order to inspect the full system. The following measures apply the core lesson from the Trivy incident: never trust an unverified, mutable source.

3.1.1 Step 1: Pin to an Exact, Verifiable Version, and Verify with GPG

```
curl -L https://github.com/aquasecurity/trivy/releases/download/v0.50.1/trivy_0.50.1_Linux_x86_64.tar.gz
curl -L https://github.com/aquasecurity/trivy/releases/download/v0.50.1/trivy_0.50.1_Linux_x86_64.tar.gz.sig
gpg --verify trivy.tar.gz.sig trivy.tar.gz
```

Table 3.1: What GPG verification protects against

Protects Against	Does Not Protect Against
Tampered binary (modified after signing)	Upstream signing key being compromised
Man-in-the-middle swapping the download	A vulnerability introduced before the release was signed
Fake or unofficial builds	Registry compromise (binary modified after upload)

This step's purpose is narrow but important: it prevents silently receiving a newer, potentially compromised release

3.1.2 Step 2: Verify Integrity Over Time, Not Just at Install

A one-time verification at install is not sufficient; the tool and its configuration could be tampered with afterward. To detect this, a baseline hash of the entire install directory is generated, not just the executable, since supporting files and configs are equally viable tampering targets:

```
find /usr/local/lynis -type f -exec sha256sum {} \; > baseline.txt
```

The full tree is then periodically re-checked against this baseline, rather than checking a single file in isolation:

```
# Generate and store a baseline hash after verified install
sha256sum /usr/local/lynis/lynis > /etc/lynis-baseline.sha256

# Periodically re-check
sha256sum -c /etc/lynis-baseline.sha256
```

If the hash ever changes without a deliberate, intentional update, this is a signal that the tool has been modified.

3.1.2.1 Limitation of This Approach

It is important to be explicit about what this control does and does not achieve. `/etc` is not inherently protected from a root-level attacker, and root compromise is precisely the threat model being defended against. An attacker with root access could modify the Lynis binary and the baseline hash file together, causing the integrity check to pass even though the tool is now malicious. As such, this hashing approach protects against accidental changes or a non-root-level compromise, but not against a fully privileged attacker.

A more robust solution would involve write-once (immutable) storage for the baseline hash. Given the scope and resource constraints of this project, this limitation was deliberately accepted, and the gap is documented here rather than left unaddressed.

3.2 Network Namespace Isolation

Even with a verified, integrity-checked binary, a residual risk remains: if Lynis were ever compromised despite these controls through a supply-chain attack identical to the Trivy incident, it would still run with full root privileges and, by default, unrestricted network access. To close this gap, Lynis is executed inside an isolated network namespace, removing network access as an available attack path regardless of what privileges the process holds.

```
ip netns add lynis-ns
```

This creates a new, empty network namespace named `lynis-ns`. By default, it has:

- No network interfaces (no `eth0`, no `wlan0`, nothing)
- No routing table
- No DNS configuration

- It is, from a networking standpoint, a completely blank slate

```
ip netns exec lynis-ns lynis audit system
```

This executes `lynisauditsystem` inside that namespace. Lynis retains full root privileges for everything it legitimately needs, reading files, checking permissions, inspecting configurations,

3.2.1 Why Root Privileges Don't Undermine This Control

Normally, root can bypass almost any restriction, which is why a simpler UID-based sandboxing approach would be ineffective here. But a network namespace is not a permission that root can override, it is a resource that simply does not exist inside the namespace. Root running inside `lynis-ns` still has zero network devices available to it. Even a fully compromised, root-privileged process has nothing to escalate into, network-wise.

3.3 Scan Results

Following the secured Lynis scan, executed within the isolated network namespace using the pinned and integrity-verified binary, the following results were obtained.

Table 3.2: Lynis scan: test count breakdown

Metric	Value
Total tests available	458
Tests performed	256
Tests skipped	202
Completion rate	~56%

The reduced completion rate is a direct, expected consequence of the network namespace isolation applied earlier: since `lynis-ns` has no network interfaces, routing table, or DNS configuration, all tests that depend on network connectivity (e.g. DNS resolution checks, remote reachability checks) are automatically skipped rather than failed.

The report flagged 4 warnings, of which 3 were assessed as false positives after manual review. The one remaining valid finding was addressed, and the underlying configuration was corrected accordingly.

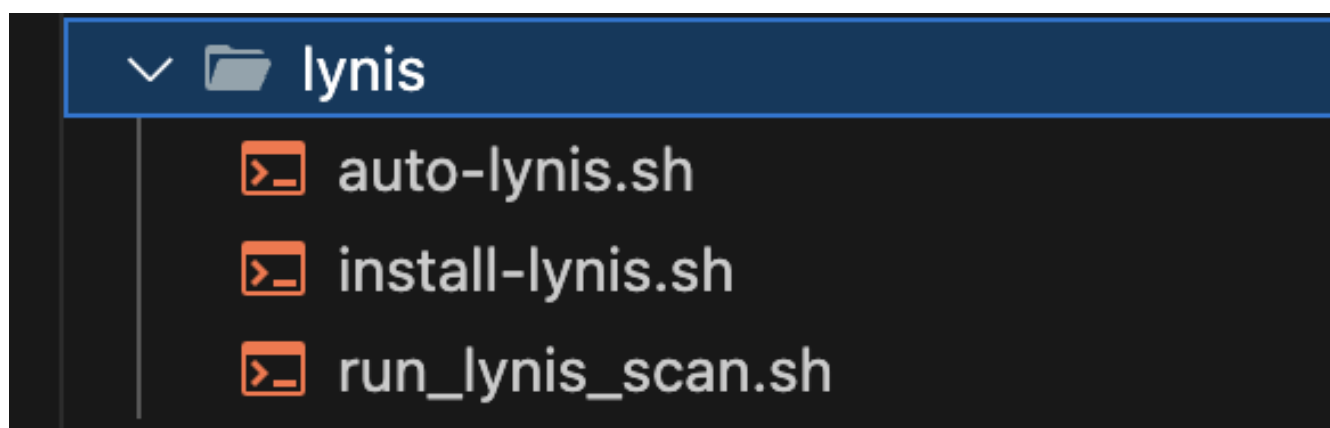


Figure 3.1: Lynis project folder structure: install-lynis.sh, run_lynis_scan.sh, auto-lynis.sh

3.3.1 Automation Scripts Overview

To operationalize the process described above, three scripts were developed.

Table 3.3: Lynis automation scripts

Script	Purpose
<code>install-lynis.sh</code>	Performs the secured installation described earlier: downloading the pinned version, verifying it via GPG, and generating the initial integrity baseline hash.
<code>run_lynis_scan.sh</code>	Runs a manual, on-demand scan inside the isolated <code>lynis-ns</code> network namespace.
<code>auto-lynis.sh</code>	Automates recurring scans via a scheduled cronjob.

3.4 Automated Security Patching: Unattended Upgrades

```
# Install and enable automatic security updates
apt install unattended-upgrades
dpkg-reconfigure unattended-upgrades
```

Once configured, Ubuntu automatically applies security patches every night, reducing the exposure window for known vulnerabilities

3.5 Fail2Ban: Brute-Force Protection

To protect the host against brute-force login attempts, Fail2Ban was installed and configured. Fail2Ban monitors authentication logs (e.g. SSH login attempts) and automatically bans source IPs that exhibit repeated failed login attempts within a defined time window, blocking them at

the firewall level for a configurable duration.

This adds a lightweight but effective layer of defense against automated credential-stuffing and brute-force attacks targeting exposed services such as SSH, complementing the OS-level hardening already in place.

3.6 kube-bench: Kubernetes Cluster Hardening Audit

A separate tool is required to assess the security posture of the Kubernetes cluster itself. For this, **kube-bench** was used, benchmarking the cluster configuration against **CIS Kubernetes Benchmark** standards.

Consistent with the approach applied to Lynis, kube-bench was also executed inside an isolated network namespace, ensuring the same containment principle is applied uniformly across scanning tools in this project: regardless of privilege level, no scanning tool is granted unrestricted network access.

3.6.1 Scan Results

```
== Summary total ==  
78 checks PASS  
4 checks FAIL  
40 checks WARN  
0 checks INFO
```

The failed and warning checks were reviewed individually and used to guide subsequent hardening work on the cluster, detailed in the sections that follow.

3.6.2 Remediation of kube-bench Findings

Among the failed checks identified by kube-bench, three key gaps were addressed:

- Audit logging, enabled at the cluster level
- Etcd encryption at rest, enabled
- TLS client certificate authentication, enabled and enforced on etcd

3.7 Verifying TLS Client Certificate Enforcement

Enabling a security control is not sufficient on its own, it needed to be actively tested to confirm it behaved as expected.

3.7.1 Step 1: Test etcd Reachability from Another Machine on the Network

```
nc -zv 192.168.1.27 2379
```


- `-z` checks only whether the port is open, without sending data
- `-v` gives verbose output, showing the result clearly

3.7.2 Step 2: Test etcd API Call

```
curl -k https://192.168.1.27:2379/version
```

This determines whether TLS client certificate authentication is actually being enforced:

- If enforcement is correct, the request should be rejected with a certificate-required error.
- If etcd instead responds with real version information, that would indicate a serious misconfiguration, etcd answering requests with no authentication at all.

Result:

```
error:0A00045C:SSL routines::tlsv13 alert certificate required
```

What this confirms: etcd rejected the connection because no valid client certificate was presented. This confirms that TLS client certificate authentication is correctly enforced on etcd.

3.8 Addressing Residual Network Exposure

While the TLS client certificate test confirmed that etcd correctly rejects unauthenticated requests, the earlier `nc` reachability test still succeeding pointed to a residual issue worth addressing on its own: the port itself remained network-reachable from any device on the LAN, independent of whether authentication was enforced correctly.

3.8.1 Is an Open, Reachable Port Actually a Problem Here?

Partially, it constitutes an exposure, not a breach. Concretely, it means:

- Any device on the LAN can confirm that etcd's port is open, useful reconnaissance information for a potential attacker
- Any device on the LAN can attempt to communicate with etcd

It does not mean:

- Anyone can actually read or write data in etcd
- Anyone can extract Secrets from this alone

3.8.2 Why It Still Matters

1. **Unnecessary attack surface.** A port that has no legitimate reason to be reachable from arbitrary machines on the LAN is exposed regardless. This is unwarranted risk, even

while correctly protected by certificates.

2. **Single point of failure.** If certificate enforcement were ever weakened, missing, or misconfigured, this open port would become immediately exploitable. The curl test confirmed certificates are currently enforced correctly, but that means this risk is mitigated by exactly one control, with no defense in depth behind it.
3. **Stolen-certificate scenario.** If a valid client certificate were ever stolen (e.g. by compromising a node where certificate files reside on disk), this open network path is precisely what would allow that stolen certificate to reach etcd from anywhere on the LAN, rather than being confined to the control-plane node itself.

3.8.3 Remediation: Restricting etcd Access at the Network Layer

To eliminate this exposure, iptables rules were added to restrict access to etcd's port to the control-plane IP only, denying all other sources:

```
sudo iptables -A INPUT -p tcp --dport 2379 -s 192.168.1.27 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 2379 -j DROP
```

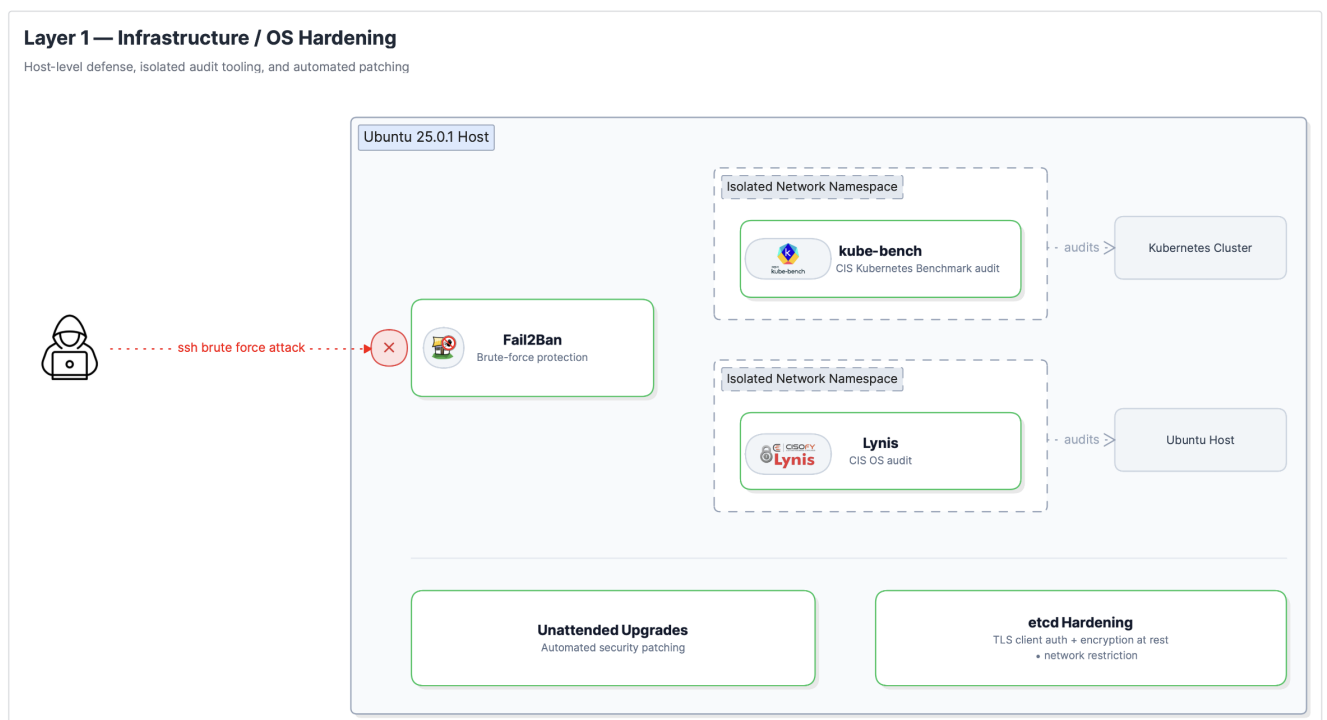


Figure 3.2: Overview of Layer 1: Infrastructure/OS hardening controls

CHAPTER 4

Layer 2: Application Workload Security

Layer 2 covers the security of what runs inside the cluster: pod-level securityContext hardening, RBAC least-privilege access control, network policies, and automated policy enforcement via **Kyverno**.

Before detailing these controls, it is necessary to first introduce the application being secured: its architecture, technology stack, and the deliberately introduced vulnerabilities used to validate that these controls work as intended.

4.1 Application Overview

The application secured in this layer, **MindGrid**, follows a microservices architecture composed of a frontend and three independent backend services.

Table 4.1: MindGrid components

Component	Language / Framework	Responsibility
Frontend	React (Vite)	User-facing client application
Auth Service	Node.js / Express	User authentication and JWT issuance
Puzzle Service	Python / Flask	Puzzle logic and answer submission
Leaderboard Service	Node.js / Express	Score tracking and leaderboard data

4.1.1 Communication

All services expose REST APIs over HTTP, routed under a common `/api` prefix. Service-to-service communication is authenticated using a static, shared `X-Service-Key` header, a design choice revisited in the RBAC and network policy sections below, where its limitations relative to per-service identity are discussed.

4.1.2 Storage

All three backend services share a single **PostgreSQL 15** database instance deployed as a StatefulSet.

4.1.3 Secrets Management

Sensitive configuration, `DB_HOST`, `JWT_SECRET`, `SHARED_SERVICE_KEY`, and related values, is managed via environment variables, layered through Kubernetes ConfigMaps and Secrets, as detailed in the deployment section below.

4.1.4 Authentication and Authorization

- User-facing auth uses stateless JSON Web Tokens (JWT).
- Service-to-service auth uses a static shared key (`SHARED_SERVICE_KEY`), sent via the `X-Service-Key` header.

4.1.5 Intentionally Introduced Vulnerabilities

To validate that the security controls in this layer function correctly under realistic conditions rather than theoretical ones, two vulnerabilities were deliberately introduced into the application:

VULNERABILITY

- **SQL Injection**, in the Puzzle Service's `submit_answer` endpoint
- **IDOR** (Insecure Direct Object Reference), in the Leaderboard Service's `/submissions/:id` endpoint

These serve as concrete test cases throughout Layers 2 and 3: each control introduced afterward is evaluated in part by whether it detects, contains, or mitigates these two flaws.

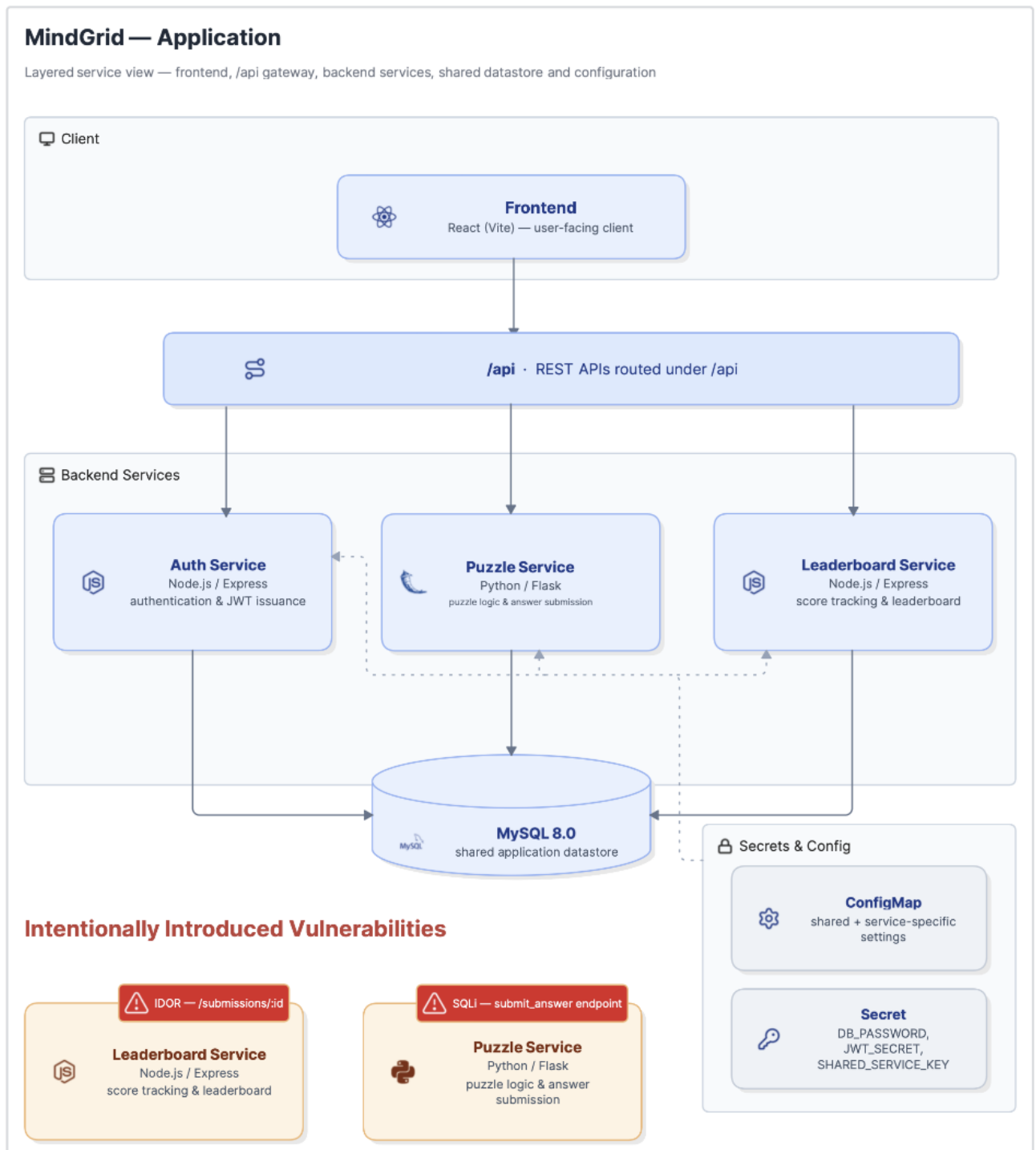


Figure 4.1: MindGrid application architecture

4.2 MindGrid Application Architecture Overview

4.2.1 Containerization Approach

Each service was first packaged into a Dockerfile with a straightforward, functional configuration. At this stage, no security hardening was applied at the container or pod level, Kubernetes

manifests would later override and enforce the relevant securityContext settings, so baseline correctness was prioritized first, with hardening addressed as a dedicated pass afterward.

4.2.2 Choosing nginx-unprivileged for the Frontend

For the frontend's static file server, `nginx-unprivileged` was used instead of the standard `nginx` image, a deliberate choice aligned with the non-root requirement enforced later at the pod level via securityContext.

Most nginx deployments are not running inside Kubernetes with `runAsNonRoot` enforced. Plain nginx running directly on a VM, or in a Docker container with no particular security hardening, has no reason to avoid root, it is the traditional default, and root works fine there. The non-root requirement is specifically a **Kubernetes security best practice**, not a general nginx requirement. In other words, `nginx-unprivileged` is not a replacement for `nginx`, it is a variant built for one specific situation: containerized environments, primarily Kubernetes, where privilege escalation is being intentionally locked down.

4.3 Cluster Node Architecture: Environment Isolation

To enforce a clear separation between development and production workloads at the infrastructure level, the cluster was structured with one control-plane node and two worker nodes, each dedicated to a specific environment.

Table 4.2: Cluster node roles

Node	Role
Control Plane	Cluster management
dev-node	Dedicated to development workloads
prod-node	Dedicated to production workloads

4.3.1 Node Labeling and Tainting

Each worker node was labeled according to its environment and tainted to prevent unintended workload placement:

```
kubectl taint nodes dev-node environment=dev:NoSchedule
```

4.3.2 Granting Access via Tolerations

To allow a specific pod to be scheduled onto `dev-node`, a matching toleration must be declared in its manifest:

```
tolerations:
```

```
- key: "environment"
  operator: "Equal"
  value: "dev"
  effect: "NoSchedule"
```

4.3.3 Automated Environment Routing with Kyverno

Relying on developers to manually add the correct toleration to every pod manifest is error-prone and easy to forget, a missing toleration simply results in a pod stuck in Pending, with the cause not always obvious at a glance.

To remove this manual step entirely, **Kyverno** was configured with a mutating policy that automatically injects the appropriate toleration based on the target namespace, so pods deployed to dev or prod are routed to their corresponding node .

4.4 Supporting Tooling

Alongside the application itself, the following tools were deployed to support the DevSecOps workflow, each exposed on a dedicated port.

Table 4.3: Supporting DevSecOps tooling

Tool	Port	Purpose
MindGrid (application)	80	The application being secured
SonarQube	9000	Static code analysis / code quality
Nexus	8081	Artifact repository
DefectDojo	8080	Vulnerability management and findings aggregation

These tools form the operational backbone referenced throughout the remaining Layer 2 and Layer 3 sections. SonarQube and Nexus support the build pipeline, while DefectDojo centralizes scan results from all stages for tracking and triage.

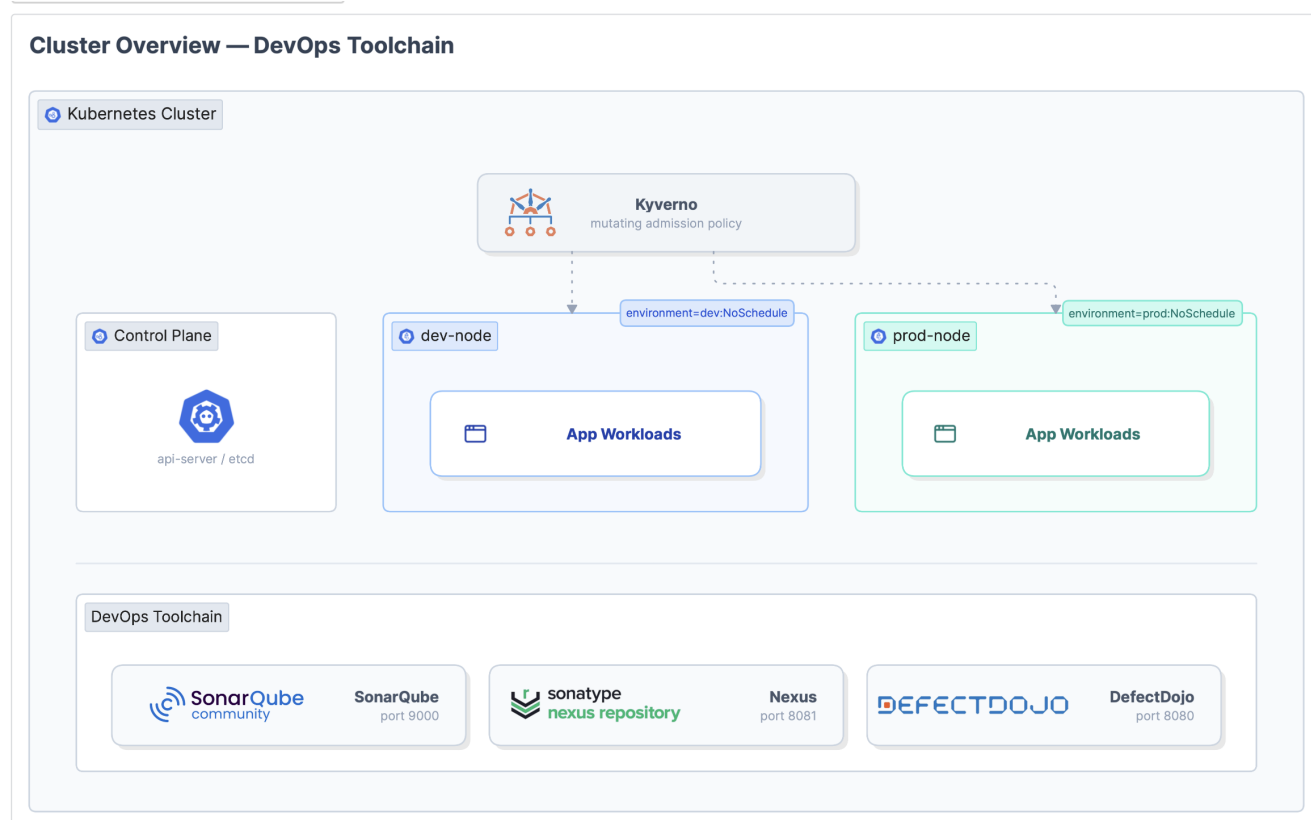


Figure 4.2: Cluster overview: DevOps toolchain

4.5 Scope of Hardening

Not every component in the cluster received the same depth of security treatment, and that is a deliberate choice rather than an oversight.

NOTE The goal of this project was never to exhaustively harden every tool deployed, some of that work is considerably more complex than the scope of a single-person project allows. Instead, the focus was on the components directly tied to the application and its pipeline, using them to learn and apply repeatable, secure patterns, the same kind of pattern already demonstrated with Lynis and kube-bench: pin the version, verify it, isolate it, scope its access.

That pattern, once understood, is transferable to any tool, which matters more than covering every tool in the cluster individually.

4.6 Kyverno Policy Suite

Kyverno was used as the cluster's policy engine to enforce and automate security best practices declaratively, rather than relying on manual review or convention. The project's Kyverno configuration is organized into seven policy files, each targeting a specific concern.

Table 4.4: Kyverno policy files

File	Purpose
<code>auto-node-routing.yaml</code>	Mutating policy: automatically injects the correct toleration based on namespace, routing pods to dev-node or prod-node
<code>kyverno-values.yaml</code>	Helm values file granting Kyverno's own controllers their own tolerations directly, see note below
<code>disallow-automount-sa-token.yaml</code>	Blocks automatic mounting of service account tokens into pods that do not explicitly require them
<code>disallow-latest-tag.yaml</code>	Rejects image references using the mutable latest tag, enforcing pinned, immutable versions
<code>enforce-pod-security-restricted.yaml</code>	Enforces the Kubernetes Pod Security Standards Restricted profile at admission time
<code>require-probes.yaml</code>	Requires liveness and readiness probes to be defined on every pod
<code>verify-image-signature.yaml</code>	Verifies container image signatures at admission, rejecting unsigned or improperly signed images

4.6.1 A Necessary Exception: Kyverno's Own Controllers

The `auto-node-routing.yaml` mutating policy handles toleration injection for application workloads deployed into dev and prod namespaces, but its scope was never designed to cover Kyverno's own controller pods, which run outside that namespace boundary. Left unaddressed, this would have left Kyverno's controllers unable to schedule onto the tainted nodes at all.

To resolve this, `kyverno-values.yaml` grants each of Kyverno's four controllers (admissionController, backgroundController, cleanupController, reportsController) their own tolerations directly in the Helm values, rather than attempting to route them through the same automated mutation mechanism used for application pods.

4.7 Kubernetes Deployment Architecture

4.7.1 Overall Architecture

The application is deployed as five workloads behind a single Gateway entry point, backed by a PostgreSQL database.

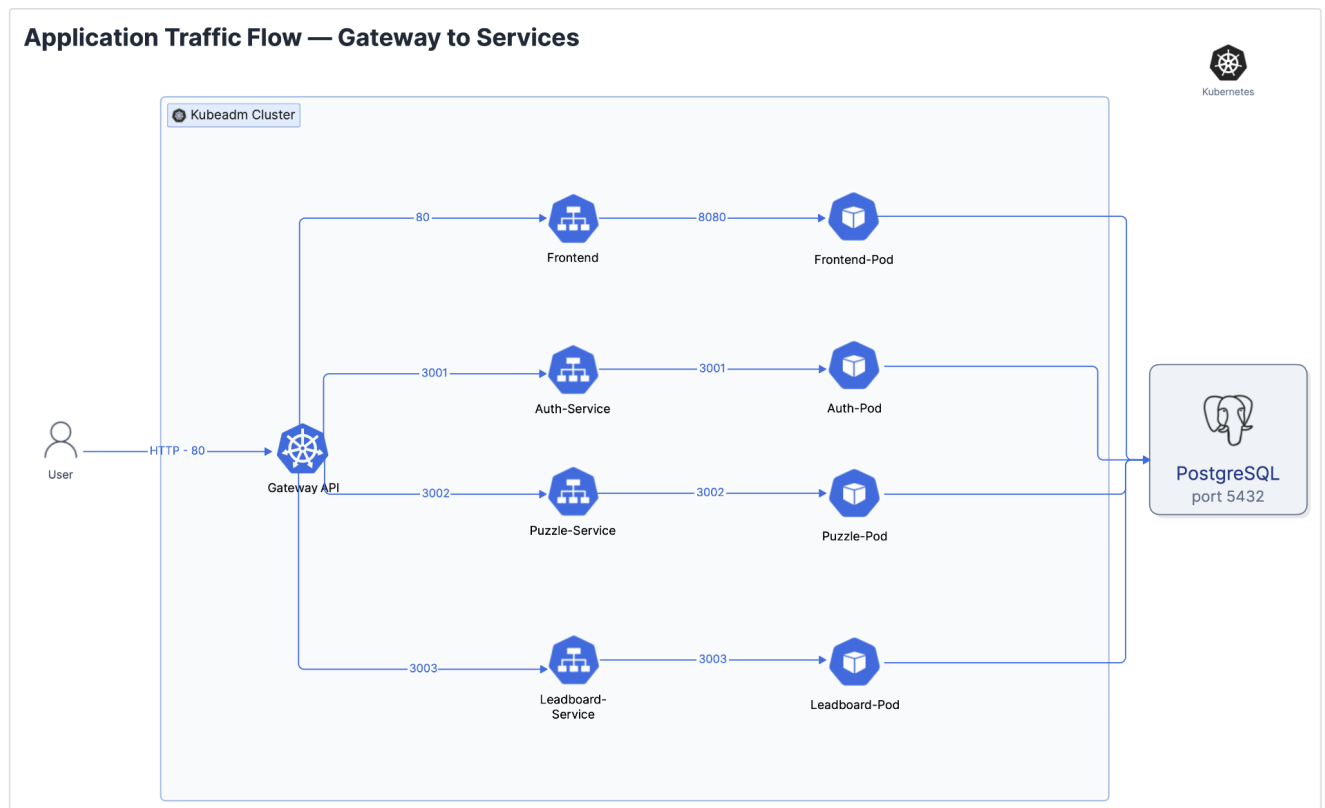


Figure 4.3: Application traffic flow: Gateway to Services

Table 4.5: Application workloads

Workload	Kind	Port	Image (pinned by digest)
auth-service	Deployment	3001	yassine123432/auth-service@sha256:95b64fe...
puzzle-service	Deployment	3002	yassine123432/puzzle-service@sha256:62c111f...
leaderboard-service	Deployment	3003	yassine123432/leaderboard-service@sha256:30141...
frontend	Deployment	8080	yassine123432/frontend@sha256:276822...
postgres	StatefulSet	5432	postgres:15

Every image reference is pinned by digest. All services are exposed exclusively as **ClusterIP**, no NodePort or LoadBalancer is used at the service level, keeping backend services unreachable from outside the cluster directly. External access is only possible through the Gateway.

4.7.2 Gateway API Layer

Files: `gateway.yaml`, `route.yaml`, `auth-login-route.yaml`, `rate-limit-policy.yaml`, `nginx-proxy.yaml`, `nginx-gateway-probe-exception.yaml`

The project uses the Kubernetes **Gateway API** (`gateway.networking.k8s.io/v1`), backed by **NGINX Gateway Fabric**, rather than a classic Ingress resource, a newer, more expressive standard that supersedes `networking.k8s.io/v1/Ingress`.

A single Gateway object (`myapp-gateway`) listens on HTTP port 80. Routing is declared through two HTTPRoute objects.

`app-route`: prefix-based path routing to the four backends.

Table 4.6: Gateway path routing

Path Prefix	Backend	Port
<code>/api/auth</code>	auth-service	3001
<code>/api/puzzle</code>	puzzle-service	3002
<code>/api/leaderboard</code>	leaderboard-service	3003
<code>/ (catch-all)</code>	frontend	80

`auth-login-route`: a dedicated route for the exact path `/api/auth/login`, isolated specifically so a rate-limiting policy can be attached to it alone.

NOTE Kyverno Policy Exception: `nginx-gateway-probe-exception.yaml` grants the `myapp-gateway-nginx` Deployment a Kyverno PolicyException, exempting it from the cluster-wide `require-probes` policy. This is a pragmatic workaround: NGINX Gateway Fabric manages its own deployment internally and does not expose configurable liveness probes through the standard Kubernetes API, so blanket-enforcing the probe policy here would simply break the gateway rather than improve its security posture.

4.7.3 Secret and Configuration Management

Files: `secrets/`

Configuration is cleanly split between non-sensitive and sensitive data.

Table 4.7: ConfigMaps (non-sensitive)

ConfigMap	Scope	Contents
<code>mindgrid-config</code>	Global (all services)	<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>ALLOWED_ORIGINS</code> , <code>FRONTEND_API_PREFIX</code>
<code>auth-service-config</code>	Auth service only	<code>PORT=3001</code>
<code>puzzle-service-config</code>	Puzzle service only	<code>PORT=3002</code>
<code>leaderboard-service-config</code>	Leaderboard service only	<code>PORT=3003</code> , <code>PUZZLE_SERVICE_URL</code>

The `PUZZLE_SERVICE_URL` value in the leaderboard config demonstrates inter-service communication via Kubernetes' internal DNS (`http://puzzle-service:3002`), rather than hardcoded IPs.

Table 4.8: Secrets (sensitive)

Secret	Scope	Keys
<code>mindgrid-secrets</code>	All backend services	<code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>JWT_SECRET</code> , <code>SHARED_SERVICE_KEY</code> , <code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code> , <code>GITHUB_TOKEN</code>
<code>postgres-secret</code>	PostgreSQL	<code>POSTGRES_USER</code> , <code>POSTGRES_PASSWORD</code> , <code>POSTGRES_DB</code>

4.7.4 Storage Layer

Files: `storage/`

StatefulSet, PostgreSQL. PostgreSQL is deployed as a StatefulSet rather than a Deployment to guarantee stable network identity (`postgres-0`, `postgres-1`, ...), stable and persistent storage via a volumeClaimTemplate, and ordered, graceful pod replacement.

Custom StorageClass, `postgres-retain`. A dedicated StorageClass defines three critical properties.

Table 4.9: postgres-retain StorageClass properties

Property	Value	Rationale
<code>provisioner</code>	<code>rancher.io/local-path</code>	Compatible with local dev clusters (Minikube, K3s, Kind)
<code>reclaimPolicy</code>	<code>Retain</code>	The PersistentVolume is not deleted when the PVC is deleted, data is preserved
<code>volumeBindingMode</code>	<code>WaitForFirstConsumer</code>	Volume is provisioned only after a pod is scheduled, ensuring local affinity

The Retain reclaim policy is a deliberate data-safety measure: accidental deletion of a PVC does not result in data loss.

Pod Anti-Affinity. The PostgreSQL StatefulSet defines a podAntiAffinity rule:

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: postgres
          topologyKey: kubernetes.io/hostname
```

This instructs the scheduler to prefer placing postgres pods on different nodes (topology key = hostname). In a future scenario involving scaling PostgreSQL replicas or running a read replica, this rule prevents two postgres pods from landing on the same physical node, avoiding a single node becoming a single point of failure for the database.

The `preferredDuringSchedulingIgnoredDuringExecution` mode is a soft constraint (weight 100): the scheduler tries hard to honor it, but will not block scheduling entirely if no alternative node is available, keeping the rule cluster-size agnostic.

4.8 From Working to Applying OWASP-Aligned Security Best Practices

At this stage, the application ran successfully end-to-end, the Gateway routed correctly to all four services, and each service reached PostgreSQL as expected. Functionally, the deployment was complete.

What followed was a deliberate second pass: taking a working configuration and systematically

applying OWASP-aligned security best practices to the Kubernetes manifests. Rather than baking hardening in from the first draft, this project intentionally separated "make it work" from "make it secure" as two distinct phases, so each hardening decision could be tested in isolation against a known-good baseline, and its effect clearly observed.

4.8.1 Rate Limiting on Login

File: `rate-limit-policy.yaml`

A `RateLimitPolicy` (`auth-login-rate-limit`) is attached to the `auth-login-route` `HTTPRoute`, deliberately isolated from the rest of the auth service so this policy applies only to the login endpoint.

Table 4.10: Rate limit policy parameters

Parameter	Value
Key	<code>\$binary_remote_addr</code> (per client IP)
Rate	5 requests/second
Burst	10
Delay	5
Reject code	429 Too Many Requests

This blocks brute-force attempts against the login endpoint at the gateway level, before traffic ever reaches the auth microservice.

Verification:

```
for i in {1..30}; do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://192.168.1.27:325.. \
    -H "Content-Type: application/json" \
    -d '{"username":"test","password":"wrong"}' &
done
wait
```

Firing 30 concurrent login attempts confirmed the policy correctly began returning 429 once the configured rate was exceeded, validating the control under realistic load rather than assuming it worked from configuration alone.

4.9 Security Baseline (Applied to All Deployments)

The following controls are applied uniformly across all four application Deployments (`auth-service`, `puzzle-service`, `leaderboard-service`, `frontend`). This consistency is intentional, it demonstrates a standardized security posture rather than ad-hoc, per-service hardening.

4.9.1 Non-Root Execution

Every pod runs as a non-root user, enforced at two levels.

Table 4.11: Non-root execution settings

Level	Setting	Value
Pod securityContext	<code>runAsNonRoot</code>	<code>true</code>
Pod securityContext	<code>runAsUser</code> / <code>runAsGroup</code>	10001 (backends), 101 (frontend nginx), 1234 (postgres)
Pod securityContext	<code>fsGroup</code>	Same as <code>runAsGroup</code> (ensures volume ownership)
Container securityContext	<code>privileged</code>	<code>false</code>
Container securityContext	<code>allowPrivilegeEscalation</code>	<code>false</code>

Setting `fsGroup` ensures mounted volumes are chown'd to the correct group, allowing the process to read/write volume data without needing root.

4.9.2 Read-Only Root Filesystem

All application containers set `readOnlyRootFilesystem:true`, preventing any process from writing to the container's root filesystem, significantly limiting the blast radius of a container escape or code injection.

To accommodate runtimes that legitimately need writable scratch space, `emptyDir` volumes are mounted at the minimal required paths only.

Table 4.12: Writable scratch space exceptions

Workload	Mount	Reason
All services	<code>/tmp</code>	General scratch / Node.js native modules
frontend	<code>/var/cache/nginx</code> , <code>/var/run</code>	Nginx cache and PID file
puzzle-service	<code>/.gunicorn</code>	Gunicorn worker heartbeat files (hardcoded path in Gunicorn source)

NOTE PostgreSQL is intentionally excluded from `readOnlyRootFilesystem:true` because it writes its Unix socket to `/var/run/postgresql` at startup, a path in the root filesystem that cannot be redirected. This exception is explicitly documented in a comment inside the StatefulSet manifest, rather than left as an unexplained inconsistency.

4.9.3 Capability Dropping

All containers drop every Linux capability:

```
capabilities:
  drop:
    - ALL
```

This follows the principle of least privilege, no capabilities are added back for any application container.

4.9.4 Seccomp Profile

All pods define:

```
seccompProfile:
  type: RuntimeDefault
```

This enables the container runtime's default seccomp profile (typically derived from Docker's default allowlist), restricting the syscalls available to the container process, a defense-in-depth measure against syscall-based exploits.

4.9.5 Image Pinning by Digest

All custom service images are referenced by SHA-256 digest rather than a mutable tag:

```
image: yassine123432/auth-service@sha256:95b64fe338b7f..
```

Why this matters: a tag like `nginx:1.25` looks fixed, but tags are mutable pointers, someone can push a new image and reassign that same tag to different content tomorrow. A deployment YAML referencing `nginx:1.25` can silently start pulling different content without a single line changing. Pinning by digest guarantees every deployment uses the exact image that was scanned and verified in the CI pipeline, eliminating tag-mutation attacks entirely.

4.9.6 Liveness and Readiness Probes

All five workloads implement both a `livenessProbe` and a `readinessProbe`.

- Application services use `httpGet` on their `/health` endpoint.
- Frontend uses `httpGet` on `/`.
- PostgreSQL uses `exec:pg_isready`, appropriate for a database with no HTTP layer.

This ensures Kubernetes can detect and restart unhealthy pods, and routes traffic only to pods that are actually ready to serve it.

4.10 Service Accounts and Identity

Files: `serviceaccount/`

Each workload has a dedicated, least-privilege ServiceAccount.

Table 4.13: Service accounts

ServiceAccount	Used by	<code>automountServiceAccountToken</code>
<code>auth-service-sa</code>	auth-service Deployment	<code>false</code>
<code>puzzle-service-sa</code>	puzzle-service Deployment	<code>false</code>
<code>leaderboard-service-sa</code>	leaderboard-service Deployment	<code>false</code>
<code>frontend-sa</code>	frontend Deployment	<code>false</code>
<code>postgres-sa</code>	PostgreSQL StatefulSet	<code>false</code>

Key design choice, `automountServiceAccountToken: false`: by default, Kubernetes automatically mounts a service account token into every pod, giving it the ability to authenticate against the Kubernetes API server. Since none of these workloads need to call the Kubernetes API, token automounting is explicitly disabled.

ServiceAccounts are also labeled with component and tier metadata (backend, frontend, database, service, presentation, data) for operational clarity and to support selector-based policy enforcement elsewhere in the cluster.

4.10.1 Verifying the Configuration

Disabling a setting isn't the same as confirming it worked — each ServiceAccount was tested directly to verify it exists, holds no roles, and genuinely has no token mounted:

```
kubectl get serviceaccount
NAME                AGE
auth-service-sa     8h
default             8h
frontend-sa         8h
leaderboard-service-sa 8h
```

```
myapp-gateway-nginx 8h
postgres-sa          8h
puzzle-service-sa    8h
```

```
kubectl auth can-i get pods --as=system:serviceaccount:default:auth-serv..
```

Output: no

Confirms the ServiceAccount holds **no RBAC permissions** — it cannot even list pods, let alone anything more sensitive.

```
kubectl exec -it auth-servic -- cat /var/run/secrets/kubernetes..
```

Output:

```
cat: /var/run/secrets/kubernetes.io/serviceaccount/token: No such file o..
command terminated with exit code 1
```

Confirms `automountServiceAccountToken:false` is genuinely effective — there is no token file present inside the pod at all, closing off the risk even if the RBAC check above had been misconfigured. This three-step verification (identity, permissions, and token absence) reflects the same "don't trust, verify" principle applied throughout the project.

4.11 Network Policies

Files: networkpolicy/

4.11.1 Design Philosophy: Default-Deny, Zero-Trust

The network policy set follows a strict default-deny, explicit-allow model,

4.11.2 Policy-by-Policy Breakdown

00, Default Deny All. *File: 00-default-deny-all.yaml*

The foundation policy. An empty `podSelector: {}` matches every pod in the namespace. Declaring Ingress and Egress as policy types with no rules attached means all traffic, in both directions, is denied by default, every subsequent policy exists purely to carve out a specific, deliberate exception to this baseline.

01, Allow Egress: DNS. *File: 01-allow-egress-dns.yaml*

Allows all pods to send DNS queries to any destination on port 53 (UDP and TCP). This is required for Kubernetes' internal service discovery, when auth-service connects to postgres, it first resolves the hostname postgres via kube-dns.

02 through 05, Allow Gateway to Application Services (Ingress). These four policies follow an identical pattern, one per service.

Table 4.14: Gateway ingress policies

Policy	Target Pod	Allowed Source	Port
02	frontend	myapp-gateway-nginx	TCP 8080
03	auth-service	myapp-gateway-nginx	TCP 3001
04	puzzle-service	myapp-gateway-nginx	TCP 3002
05	leaderboard-service	myapp-gateway-nginx	TCP 3003

Each policy locks down the ingress of one service so that only the NGINX Gateway data-plane pods can reach it, no other pod, including another application service, can initiate a direct connection.

Key security implication: the frontend cannot directly call auth-service or puzzle-service. All traffic is forced through the gateway, ensuring routing rules and rate-limiting policies cannot be bypassed at the network level, no matter what an application-layer bug might otherwise permit.

06, Allow Services to PostgreSQL (Ingress on Postgres). *File: 06-allow-services-to-postgres.yaml*

Controls who can reach PostgreSQL. Only the three backend services can connect on port 5432. The frontend is explicitly excluded, it has no legitimate reason to access the database directly, and this policy enforces that at the network layer, independently of whatever access control exists at the application level.

07, Allow Services to PostgreSQL (Egress from Each Service). *File: 07-allow-services-egress-to-postgres.yaml*

Contains three NetworkPolicy objects in one YAML file:

```
allow-auth-egress-to-postgres -> auth-service egress -> postgres:5432
allow-puzzle-egress-to-postgres -> puzzle-service egress -> postgres:5432
allow-leaderboard-egress-to-postgres -> leaderboard-service egress -> postgres:5432
```

Why this is needed alongside 06: Kubernetes NetworkPolicy requires both sides of a connection to explicitly permit it. Policy 06 opens Postgres's ingress door from the services; policy 07 opens each service's egress door toward Postgres. Both must exist simultaneously for the TCP connection to succeed, the correct bidirectional pattern for pod-to-pod communication under a default-deny baseline.

08, Allow Kubelet Health Probes (Ingress). *File: 08-allow-kubelet-health-probes.yaml*

1

```
podSelector: {}      # all pods
ingress:
  - from:
    - ipBlock: { cidr: 192.168.0.190/32 }
    - ipBlock: { cidr: 192.168.0.185/32 }
    - ipBlock: { cidr: 192.168.0.154/32 }
```

Liveness and readiness probes are sent by the kubelet process running on the node host, not from another pod, so probe traffic arrives from the node's host-network IP, which cannot be matched with a podSelector.

An ipBlock selector is required instead, pinned to the exact IPs of the three cluster nodes using /32 (single-host) CIDRs.

09, Allow Gateway to Application Services (Egress from Gateway). *File: 09-allow-gateway-egress-to-services.yaml*

```
podSelector:
  matchLabels:
    app.kubernetes.io/name: myapp-gateway-nginx
egress:
  - to: [frontend:8080]
  - to: [auth-service:3001]
  - to: [puzzle-service:3002]
  - to: [leaderboard-service:3003]
```

The egress counterpart to policies 02 through 05. Those open each service's ingress from the gateway; this policy opens the gateway's egress toward all four services, both sides must be open for traffic to actually flow.

10, Allow Gateway to NGF Controller (Cross-Namespace gRPC). *File: 10-allow-gateway-egress-to-nginx-controller.yaml*

```
podSelector:
  matchLabels:
    app.kubernetes.io/name: myapp-gateway-nginx
egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: nginx-gateway
      ports: [TCP:8443]
```

The only cross-namespace egress policy in the entire set. NGINX Gateway Fabric has a split architecture:

data-plane pods run in the dev namespace serving real application traffic, while the controller

pod runs in nginx-gateway, pushing configuration updates to the data-plane. They communicate over gRPC on port 8443 (named agent-grpc on the controller side). Without this policy, default-deny plus namespace isolation would sever this internal configuration channel, leaving the gateway unable to receive routing updates.

`kubernetes.io/metadata.name` is the built-in, immutable namespace-name label, the canonical, most precise way to target a specific namespace in a namespaceSelector.

11, Deny Cross-Namespace Traffic. File: `11-deny-cross-namespaces.yaml`

```
podSelector: {}
ingress:
  - from:
    - podSelector: {} # same-namespace pods ONLY
egress:
  - to:
    - podSelector: {} # same-namespace pods ONLY
```

A `podSelector: {}` inside a from/to clause, without an accompanying namespaceSelector, matches only pods in the same namespace. This policy explicitly permits same-namespace traffic while implicitly blocking everything else.

Combined with policy 10 (which carves out the one legitimate cross-namespace channel), the net result is: all cross-namespace communication is denied except the NGF gRPC controller channel.

4.11.3 Connectivity Matrix

Table 4.15: Network policy connectivity matrix

Source → Destination	frontend	auth	puzzle	leaderboard	postgres	NGF Ctrl
Gateway (data-plane)	Allow :8080	Allow :3001	Allow :3002	Allow :3003	Deny	Allow :8443
frontend	—	Deny	Deny	Deny	Deny	Deny
auth-service	Deny	—	Deny	Deny	Allow :5432	Deny
puzzle-service	Deny	Deny	—	Deny	Allow :5432	Deny
leaderboard-service	Deny	Deny	Deny	—	Allow :5432	Deny
postgres	Deny	Deny	Deny	Deny	—	Deny
kubelet node IPs	Probes	Probes	Probes	Probes	Probes	—
DNS (all → :53)	Allow	Allow	Allow	Allow	Allow	—

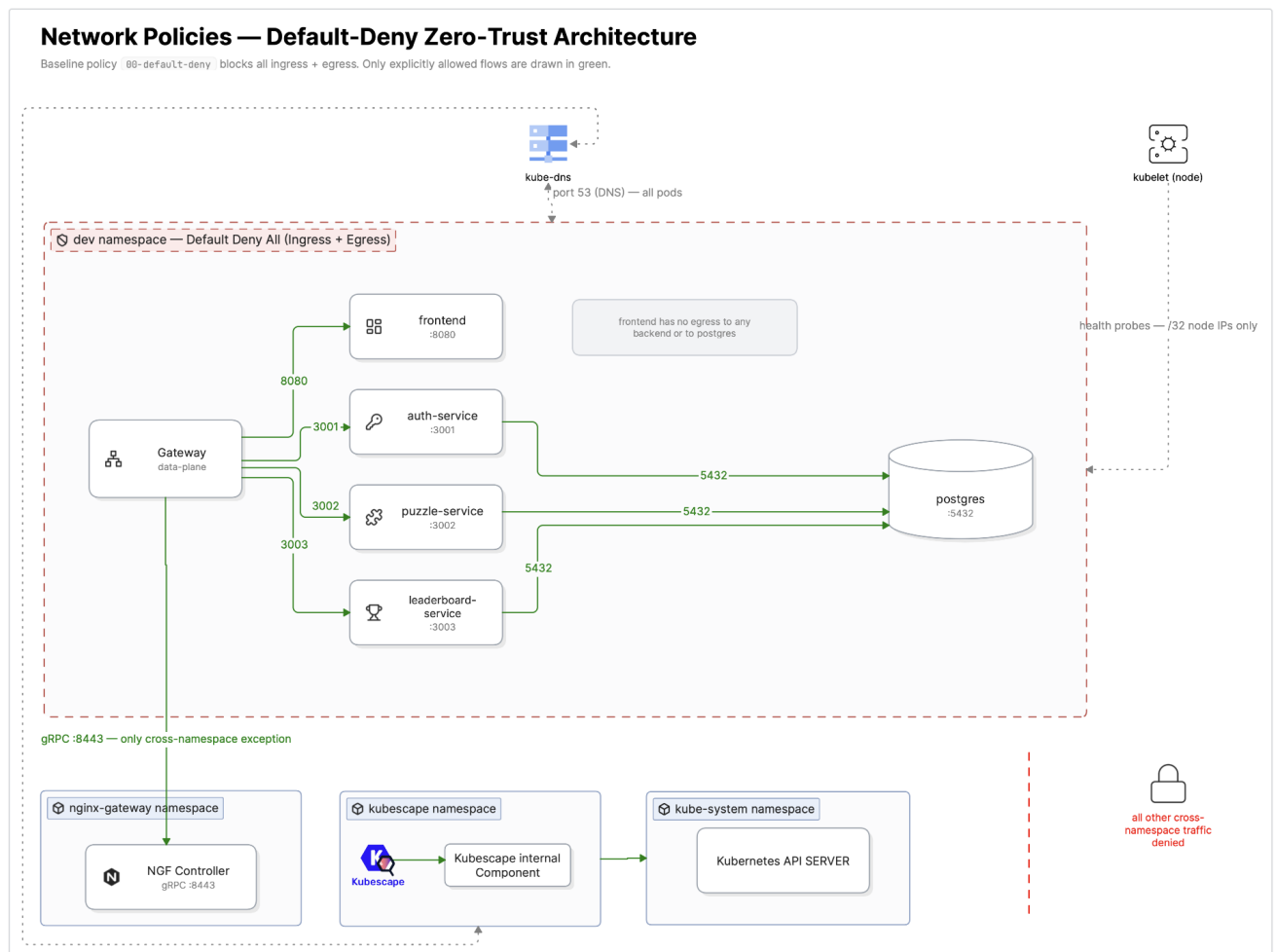


Figure 4.4: Network policies: default-deny zero-trust architecture

4.12 Kubescape: Runtime Compliance Scanning

Files: `kubescape/`

Following the same principle applied to Lynis and kube-bench in Layer 1, every scanning tool must itself be scrutinized before being trusted. **Kubescape** and its operator were installed to continuously assess the cluster's compliance posture, and were then subjected to their own security investigation before being fully trusted with cluster-wide access.

4.12.1 The Question: Can Kubescape Exfiltrate Data?

Kubescape's compliance checks require broad read access across the cluster, including, in many cases, the ability to inspect Secrets to determine whether they are referenced insecurely. This naturally raises a legitimate concern: if Kubescape (or a compromised version of it) can read every secret in the cluster, what actually prevents that data from leaving?

Step 1, inspect pod configurations for signs of an external destination:

```
kubectl get pods -n kubescape -o yaml | grep -i "server\\|backend\\|account\\|url"
```

This command is two steps chained by a pipe.

- `kubectl get pods -n kubescape -o yaml` retrieves the full raw configuration of every pod in the kubescape namespace, rather than the usual summary table. This dumps every environment variable, mounted volume, container argument, and image reference, the complete blueprint of each pod.
- `grep -i "server\|backend\|account\|url"` passes that output to grep, which scans for any line containing server, backend, account, or url (case-insensitive).

The goal was to surface any hint of an external destination hidden in the pod configs, for example a `KUBESCAPE_SERVER_URL` pointing to an external cloud endpoint, a backend field referencing something outside the cluster, or an API token tied to a third-party service.

4.12.2 Evaluating Candidate Controls

Several potential mitigations were considered and evaluated against the actual threat, not all of them were effective.

Table 4.16: Candidate exfiltration controls evaluated

Control	Stops Exfiltration?	Why
Egress NetworkPolicy	Yes, the real answer	Even if malicious code runs inside the pod and reads every secret in memory, it physically cannot send that data anywhere if egress is blocked. This is the only control that stops the actual exfiltration step.
Root / read-only secrets file	Barely helps	If Kubescape's pod has RBAC permission to get/list secrets via the Kubernetes API, which it needs in order to scan them, file permissions on disk are irrelevant, since it is not reading a file, it is calling the API.
Encrypting secrets at rest (etcd encryption)	No protection here	Etcd encryption protects against someone reading etcd's disk files directly. But Kubescape reads secrets through the Kubernetes API, which automatically decrypts them for authorized callers.

The key insight: any code running inside the Kubescape pods, legitimate or malicious, has API-level permission to read every secret in the cluster, assuming a ClusterRoleBinding across all namespaces, which is the typical setup. This is by design: Kubescape's compliance checks include things like "is this secret referenced insecurely," which sometimes requires inspecting the value itself. Given that read access is unavoidable, the only meaningful control left is

preventing the exfiltration of whatever is read, which is exactly what an egress policy does.

4.12.3 What Each Component Actually Needs

Before writing the egress policy, each Kubescape component's real network requirements were mapped out, so the policy could be scoped precisely rather than opened broadly.

Table 4.17: Kubescape component network requirements

Component	Talks To	Why
operator, kubescape, kubevuln	Kubernetes API server only	They fetch pod/secret/config data through the API, not by connecting to pods directly
node-agent	Local node's kubelet + container runtime (same node)	Inspects the node it runs on, processes, files, runtime behavior, entirely locally
storage	API server (aggregated API)	Serves data back to other Kubescape components via the API

This mapping directly informed the egress NetworkPolicy written for the kubescape namespace: each component is permitted to reach only the Kubernetes API server (and, for node-agent, its local node), with all other egress, including any path to an external endpoint, denied by default.

4.12.4 Final Compliance Score

Following the egress restriction and the broader hardening work applied throughout Layer 2, a final Kubescape compliance scan was run against two supported frameworks.

Table 4.18: Kubescape compliance scores

Framework	Score
MITRE ATT&CK	85.89%
NSA/CISA Kubernetes Hardening	88.30%

The compliance score is calculated by weighing control failures against the number of checks defined in each framework, remediating individual controls, or explicitly documenting accepted exceptions in the cluster baseline, is what moves this score.

CHAPTER 5

Layer 3: CI/CD Security

Layer 3 covers the security of the software supply chain itself: image vulnerability scanning, image signing, SBOM (Software Bill of Materials) generation and attestation, secrets management, branch protection rules, secret-leak detection, and dynamic application security testing.

5.1 Branching Strategy: Trunk-Based Development

The pipeline adopts a **Trunk-Based Development** (TBD) strategy rather than a long-lived develop/main split. In TBD, all feature work merges frequently into a single shared branch, main, instead of accumulating on a separate integration branch. There is no develop branch in this project: main itself is what maps directly to the dev environment.

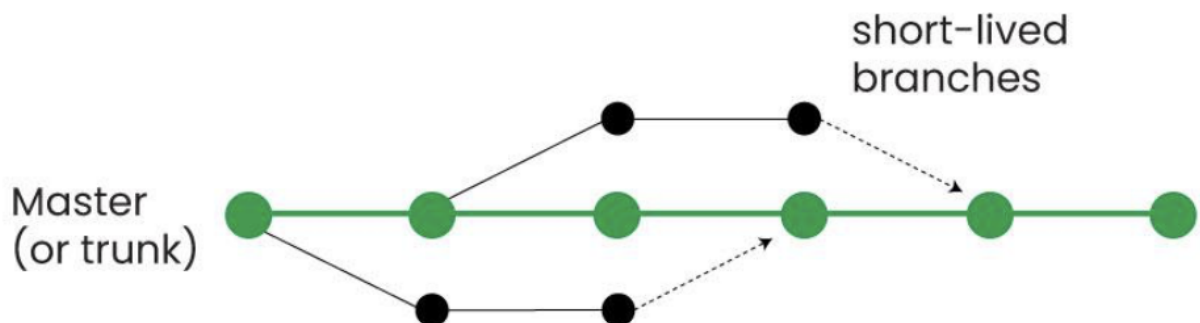


Figure 5.1: Trunk-based development: short-lived branches merging frequently into main

5.2 The Deployment Flow

A developer checks out a new branch from main (`feature/xyz`) and pushes it to the remote. Opening a pull request against main triggers the CI workflow, build, scan, sign, and the results are attached directly to the PR for review. Once the pull request is approved and merged, the change lands on main, which deploys to the dev namespace.

Promotion to production is handled as a separate, explicitly gated stage: a production environment is configured in the CI platform with a required manual approval. When that approval button is pressed, a distinct deployment job is triggered, deploying to the prod namespace.

Table 5.1: Deployment stages, triggers, and gates

Stage	Trigger	Gate
Dev deployment	Merge to main (after PR approval)	PR review/approval only
Prod deployment	Manual approval on the production environment	Explicit human approval required

Why this design matters:

- The pull request itself acts as the first checkpoint, code review plus automated build/scan/sign results are visible together before anything merges into main.
- Merging to main is what triggers deployment to dev, keeping the feedback loop tight, since every approved change is immediately validated in a running environment.
- Production is intentionally decoupled from the merge event itself. It requires a second, independent human decision, approving the production environment, before its own deployment job runs.

5.3 Securing the CI/CD Pipeline

5.3.1 Branch Protection Rules

To prevent unreviewed or unverified code from reaching main, and by extension, the dev environment, the following branch protection rules were enforced on the repository.

- **Require a pull request before merging.** No direct pushes to main are permitted; every change must go through review.
- **Require status checks to pass before merging.** The CI pipeline's build, scan, and sign steps must complete successfully before a PR is mergeable.
- **Require branches to be up to date before merging.** A PR must be rebased against the latest main before merging, preventing outdated code from silently overwriting recent changes.
- **Dismiss stale pull request approvals when new commits are pushed.** Any new commit invalidates a prior approval, ensuring reviewers always approve the actual code being merged.
- **Require review from Code Owners.** Changes to designated areas of the codebase must be reviewed by their assigned owners, not just any contributor.
- **Require conversation resolution before merging.** All open review comments must be resolved, preventing unaddressed concerns from being silently merged away.

- **Do not allow bypassing the above settings.** These rules apply to everyone, including repository administrators, closing off the most common real-world bypass path for branch protection.

These rules ensure that every change reaching main has been reviewed, is current, has passed automated verification, and cannot be pushed around the process, regardless of the pusher's privilege level.

5.3.2 GitHub Deployer: RBAC Identity for the Pipeline

Files: `github-deployer/`

The `github-deployer` folder contains three files that together form the RBAC identity used by the GitHub Actions CI/CD pipeline to authenticate and deploy to the cluster.

Two independent identities, not one shared identity. The same ServiceAccount definition is created twice, once in the dev namespace, once in prod, resulting in two separate identities, each with its own token.

`deployer-role.yaml` defines a namespaced Role (deliberately not a ClusterRole), applied identically in both dev and prod.

Table 5.2: Deployer Role permissions

API Group	Resources	Verbs
<code>apps</code>	<code>deployments, statefulsets</code>	Full CRUD
<code>"" (core)</code>	<code>pods, services, configmaps, secrets</code>	Full CRUD
<code>"" (core)</code>	<code>pods/portforward</code>	<code>create</code>

Why namespace-scoped RBAC, specifically: because permissions are granted via Role/RoleBinding rather than ClusterRole/ClusterRoleBinding, each ServiceAccount's authority is confined entirely to the namespace it lives in. This is precisely why two independent ServiceAccounts are required, one for dev, one for prod, rather than a single shared identity used across both.

This separation is a deliberate security boundary, not incidental duplication: a leaked dev token can never be used to touch prod, because the dev ServiceAccount simply has no RBAC permissions defined in the prod namespace at all.

5.4 The DevSecOps Pipeline

The CI/CD pipeline follows the security best practices outlined in the **OWASP Top 10 and the OWASP DevSecOps Guideline v0.2**, and fires on pull requests targeting main that touch the application services, the `k8s/` manifests, or the workflow definition itself.

5.4.1 Pipeline Stages

The pipeline runs as a strict sequence of gated stages, with parallelism only where stages are genuinely independent.

- Stage 0: Pre-Commit Secret Scanning (Local Hooks)** **Talisman** runs as a local pre-commit hook, scanning staged changes for credentials before commits are generated.
- Stage 1: Parallel Pre-Build Security Verification** Three independent checks run concurrently and gate the build pipeline:
- **Gitleaks**: Scans Git history for secrets and tokens, pushing findings directly to **DefectDojo**.
 - **SonarQube**: Static Application Security Testing (SAST) and code quality analysis.
 - **Snyk**: Software Composition Analysis (SCA) for third-party dependency vulnerabilities.
- Stage 2: Matrix Container Build and Vulnerability Scanning** The four services are built concurrently and audited with **Trivy** for **HIGH** and **CRITICAL** vulnerabilities, with scan reports pushed to **DefectDojo**.
- Stage 3: Registry Push and Digest Extraction** Images are pushed to Docker Hub, and their immutable SHA-256 digest is saved to `digest.txt` for subsequent steps.
- Stage 4: Keyless Signing and SBOM Attestation** Images are signed keylessly with **Cosign** via **Sigstore/Rekor**, while **Syft** generates a cryptographically signed **CycloneDX** SBOM attestation.
- Stage 5: Dev Deployment and Rollout Verification** Workloads are applied to the `dev` namespace, enforcing a health gate on `readyReplicas==desiredReplicas`.
- Stage 6: Dynamic Application Security Testing (DAST)** **OWASP ZAP** executes dynamic penetration tests against running dev endpoints via port-forward, routing reports to **DefectDojo**.
- Stage 7: Production Promotion via Approval Gate** Deployment to `prod` requires explicit manual sign-off in the CI platform, followed by automated rollout validation.

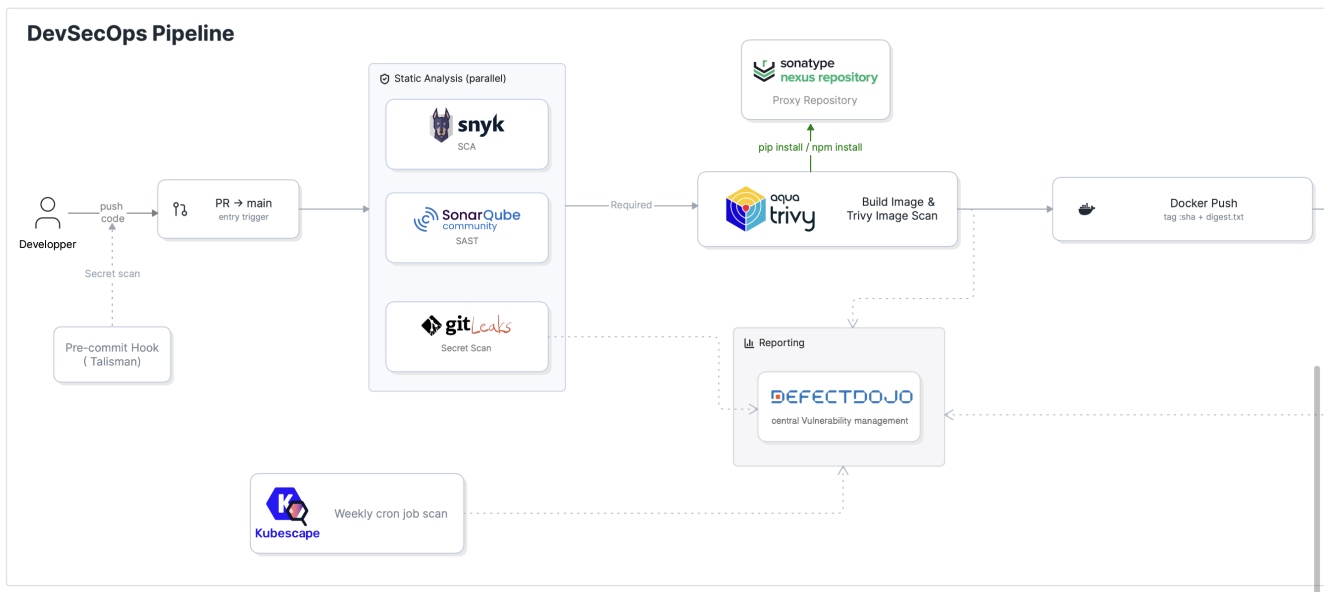


Figure 5.2: DevSecOps pipeline diagram: full sequence from PR to production deployment : Part 1

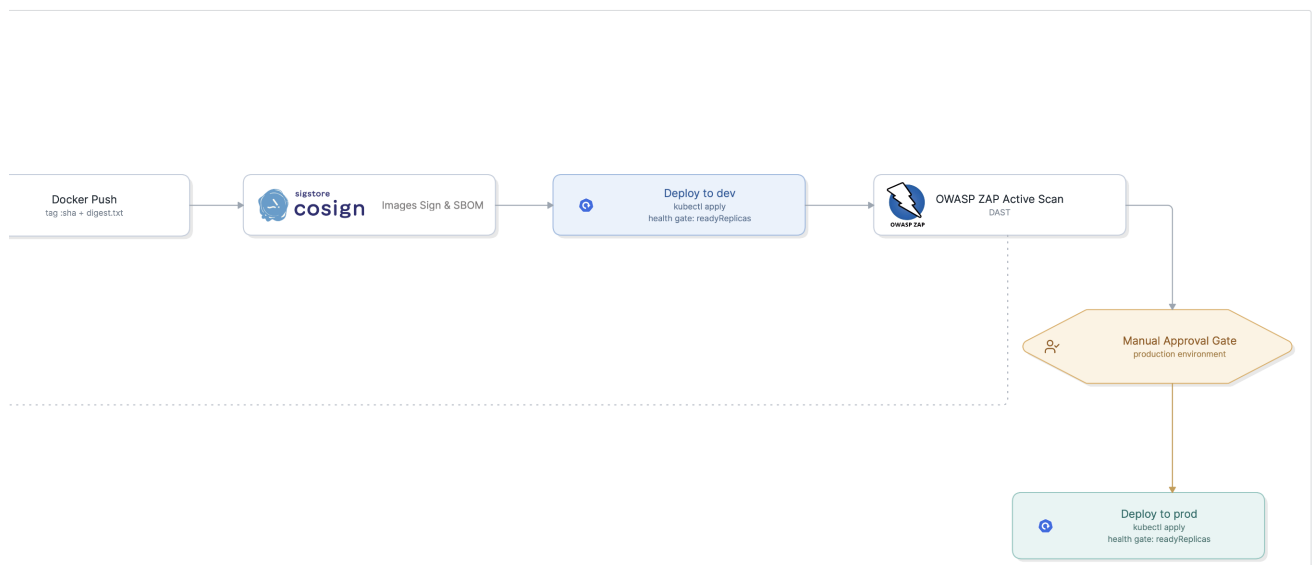


Figure 5.3: DevSecOps pipeline diagram: full sequence from PR to production deployment : Part 2

5.4.2 Security Tools Summary

Table 5.3: Security tools used across pipeline stages

Stage	Tool	Category	Reports to
gitleaks-scan	Gitleaks	Secret detection in Git history	DefectDojo
sonarqube-scan	SonarQube	SAST, code quality & vulnerabilities	SonarQube dashboard
snky-scan	Snyk	SCA, dependency vulnerabilities	Snyk dashboard
build-and-scan	Trivy	Container image scan (HIGH/CRITICAL)	DefectDojo
cosign-sign	Cosign (keyless)	Image signing via Sigstore/Rekor	Transparency log
cosign-sign	Syft	SBOM generation (CycloneDX)	Attached to image
zap-active-scan	OWASP ZAP	DAST, active attack simulation	DefectDojo

5.4.3 Key Security Design Points

- **Fail-fast ordering.** Secret scanning, SAST, and SCA all run first, in parallel, and gate everything downstream. If any fails, the build never starts, no vulnerable or secret-leaking code ever reaches a container image.
- **DefectDojo as central aggregator.** Gitleaks, Trivy, and ZAP all push results to the same DefectDojo instance via its REST API, with deduplication and engagement tracking enabled on every import, giving a single pane of glass across otherwise disconnected tools.
- **Cosign keyless signing.** Images are signed using the GitHub Actions OIDC identity directly, with no long-lived signing key to manage or leak. The signature is recorded in the public Rekor transparency log, making every signature independently auditable.
- **SBOM attached to the image.** Syft's CycloneDX SBOM is both attached to the image via `cosignattach` and separately signed as a cosign attestation, producing a verifiable software bill of materials cryptographically tied to the exact image digest, not just associated with it by convention.
- **Health gate after every deploy.** Both `deploy-dev` and `deploy-prod` explicitly verify `readyReplicas==desiredReplicas` post-rollout, failing the pipeline outright if the deployment comes up unhealthy.

- **Production requires a human.** `deploy-prod` declares `environment:production`, which GitHub enforces as a manual approval gate before the job can run
- **ZAP runs against the real, live environment.** The active scan port-forwards directly into the running frontend service in the cluster, testing the actual deployed application rather than a mock or staging replica, using ZAP's full active-attack scan mode.

5.5 Refining the ZAP Strategy: Authenticated vs. Unauthenticated Scanning

The OWASP ZAP integration went through a deliberate investigation rather than being adopted at face value. The Automation Framework was first used to manually validate the auth endpoint's behavior, before relying on the automated scan.

Running a standard full active scan directly against the gateway, however, failed to detect either of the two intentionally introduced vulnerabilities, the SQL injection in Puzzle Service and the IDOR in Leaderboard Service. The reason became clear on investigation: both vulnerable endpoints require an authenticated session to reach, and an unauthenticated scan simply never gets far enough into the application to exercise them.

This led to a concrete methodology change: the pipeline now runs two separate active scans, one unauthenticated and one authenticated, rather than a single pass. This mirrors real-world attacker behavior more accurately,

5.6 Scheduled Compliance Reporting

Beyond the per-PR pipeline, a weekly cron job runs Kubescape independently of any code change, with results automatically reported to DefectDojo, ensuring compliance drift is caught on a fixed cadence, not only when new code happens to be merged.

5.7 Nexus as a Dependency Proxy

Nexus is used as a proxy repository during the Docker build process, so all library and package downloads are routed through it rather than pulled directly from public registries at build time, reducing exposure to upstream registry outages or tampering, and giving a single point of control and visibility over every dependency pulled into a build.

CHAPTER 6

Conclusion and Future Work

As noted in the introduction, certain areas of hardening were deliberately left out of scope given the resource constraints. These are documented here as concrete future work rather than omissions.

- **Runtime threat detection with Falco.** Would provide real-time detection of anomalous syscall behavior inside running containers, complementing the static and admission-time controls already in place.
- **Centralized audit logging.** A dedicated log aggregation pipeline (e.g. the ELK stack or Loki) would allow correlation of events across Layer 1, Layer 2, and Layer 3 tooling.
- **Broader observability tooling.** Metrics and tracing across the cluster and pipeline would improve both operational visibility and the ability to detect security-relevant anomalies over time.